



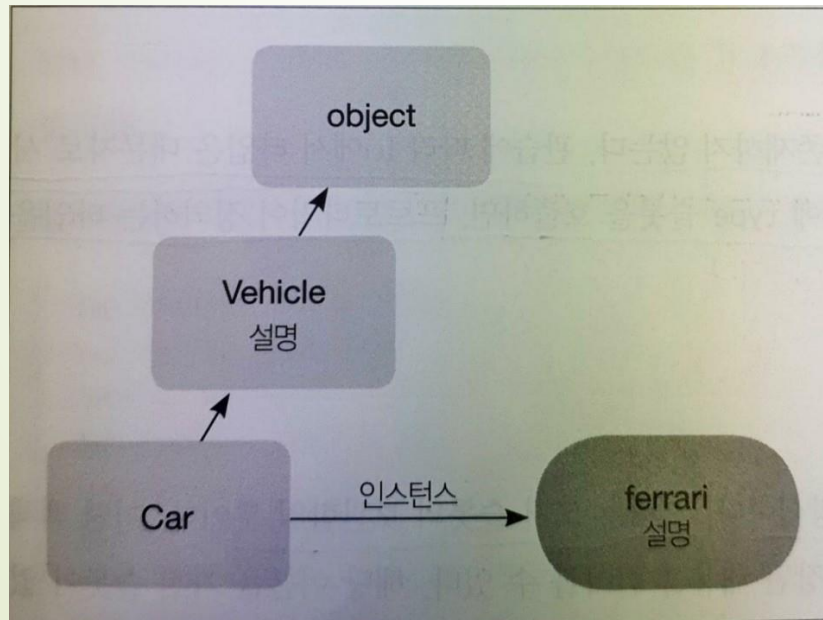
웹 시스템 개발 1

프로토타입

프로토타입이란?

- ▶ Lua, Io, Javascript는 프로토타입 기반 언어
- ▶ 모든 객체가 클래스의 인스턴스가 아닌 다른 객체의 복제물

C# 언어의 예



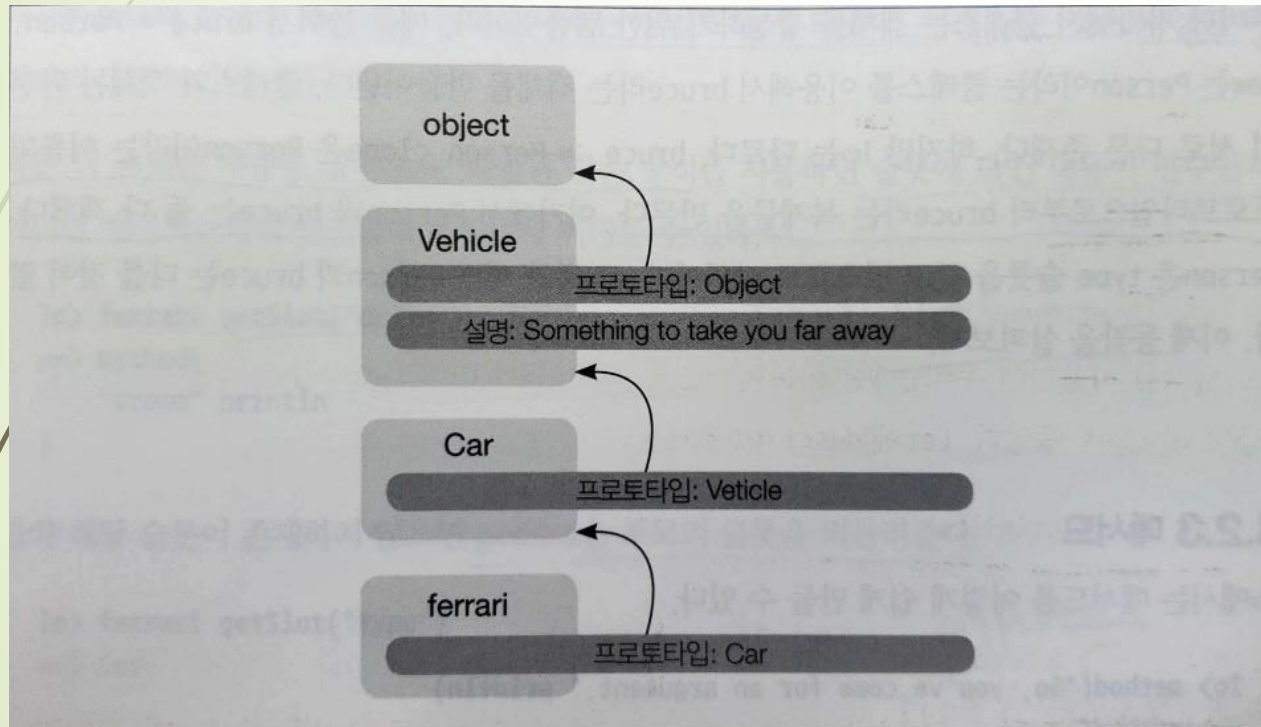
```
class Vehicle
{
}

class Car : Vehicle
{
    public void drive()
    {
        Console.WriteLine("Vrrrom");
    }
}

class MainApp
{
    static void Main(string[] args)
    {
        Car ferrari = new Car();
        ferrari.drive();
    }
};
```

프로토타입이란?

lo 언어의 예



```
lo 20110905
lo> Vehicle := Object clone
==> Vehicle_0x2e1c4e0:
    type = "Vehicle"

lo> Car := Vehicle clone
==> Car_0x2e22c00:
    type = "Car"

lo> ferrari := Car clone
==> Car_0x2e26380:

lo> ferrari proto
==> Car_0x2e22c00:
    type = "Car"

lo> Car proto
==> Vehicle_0x2e1c4e0:
    type = "Vehicle"

lo> Car drive := method("Vroom" println)
==> method(
    "Vroom" println
)

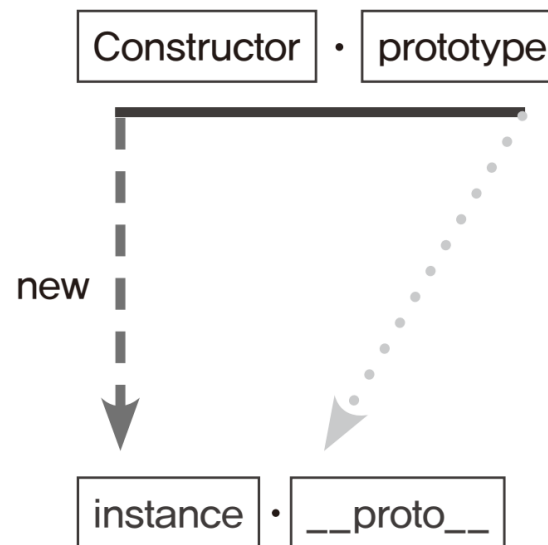
lo> ferrari drive
Vroom
==> Vroom
```

프로토타입이란?

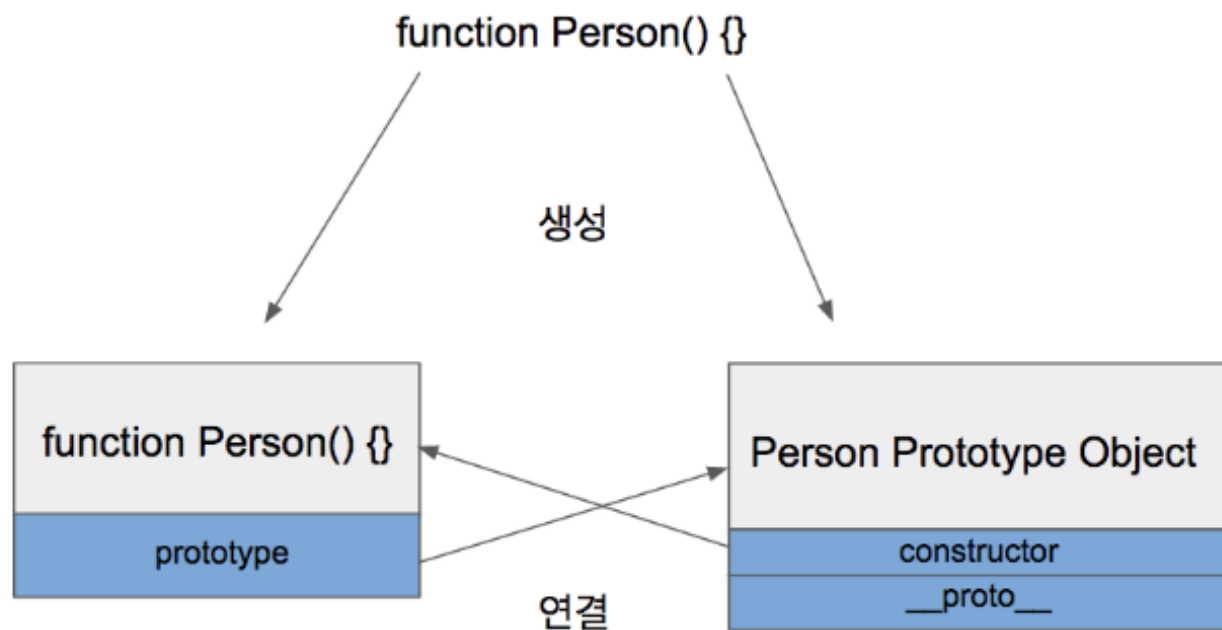
코드 6-1

```
var instance = new Constructor();
```

그림 6-2 프로토타입 도식(2)



프로토타입이란?



```
> function Person() {}  
< undefined  
  
> Person.prototype  
< ▼ Object {} ⓘ  
  ▶ constructor: function Person()  
  ▶ __proto__: Object
```

```
1  
2  function Person() {}  
3  var wonvin = new Person();  
4  
5  var obj = {};  
6  var obj = new Object();  
7
```

프로토타입이란?

```
1
2 function Person() {}
3 Person.prototype.eyes = 2;
4 Person.prototype.nose = 1;
5
6 var kim = new Person();
7
8 console.log(kim.__proto__ === Person.prototype);
9 console.log(Object.getPrototypeOf(kim) === Person.prototype);
10
11 console.log(kim.__proto__.eyes);
12 console.log(kim.__proto__.nose);
13 console.log(kim.eyes);
14 console.log(kim.nose);
15
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe ex.js

true

true

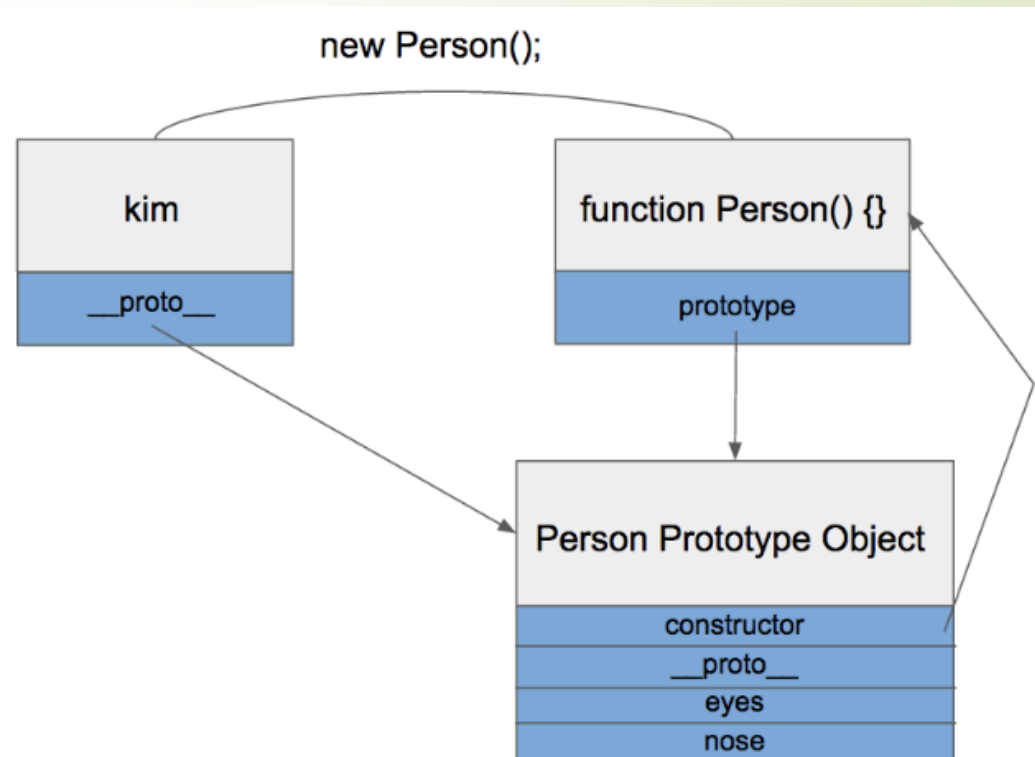
2

1

2

1

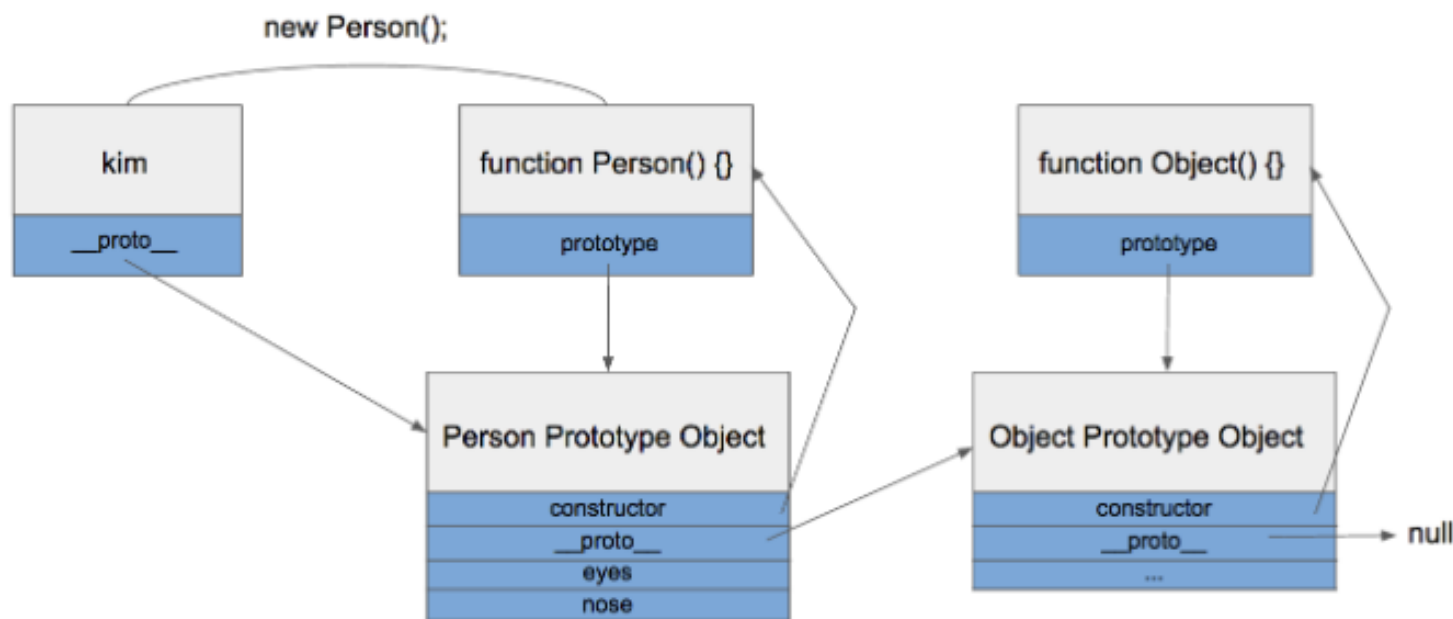
```
> function Person() {}
< undefined
> Person.prototype.eyes = 2;
< 2
> Person.prototype.nose = 1;
< 1
> Person.prototype
< ▼ Object {eyes: 2, nose: 1} ⓘ
  ► constructor: function Person()
    eyes: 2
    nose: 1
  ► __proto__: Object
```



프로토타입 이란?

```
> Object.prototype
< ▼ Object {} ⓘ
  ▶ __defineGetter__: function __defineGetter__()
  ▶ __defineSetter__: function __defineSetter__()
  ▶ __lookupGetter__: function __lookupGetter__()
  ▶ __lookupSetter__: function __lookupSetter__()
  ▶ constructor: function Object()
  ▶ hasOwnProperty: function hasOwnProperty()
  ▶ isPrototypeOf: function isPrototypeOf()
  ▶ propertyIsEnumerable: function propertyIsEnumerable()
  ▶ toLocaleString: function toLocaleString()
  ▶ toString: function toString()
  ▶ valueOf: function valueOf()
  ▶ get __proto__: function get __proto__()
  ▶ set __proto__: function set __proto__()

> kim.toString();
< "[object Object]"
```



```
1
2  function Person() {}
3  Person.prototype.eyes = 2;
4  Person.prototype.nose = 1;
5
6  var kim = new Person();
7  var park = new Person();
8
9  console.log(kim.eyes);
10 console.log(kim.toString());
11
```


프로토타입이란? (주의 사항)

```
1
2  var Person = function (name) {
3    |   this._name = name;
4  };
5
6  Person.prototype.getName = function() {
7    |   return this._name;
8  };
9
10 var kim = new Person("Kim");
11 console.log(kim.__proto__.getName());
12 console.log(kim.getName());
13
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe ex.js
undefined
Kim

프로토타입이란?

```
1
2 var arr = [1, 2];
3 console.dir(arr);
4 console.dir(Array);
5
```

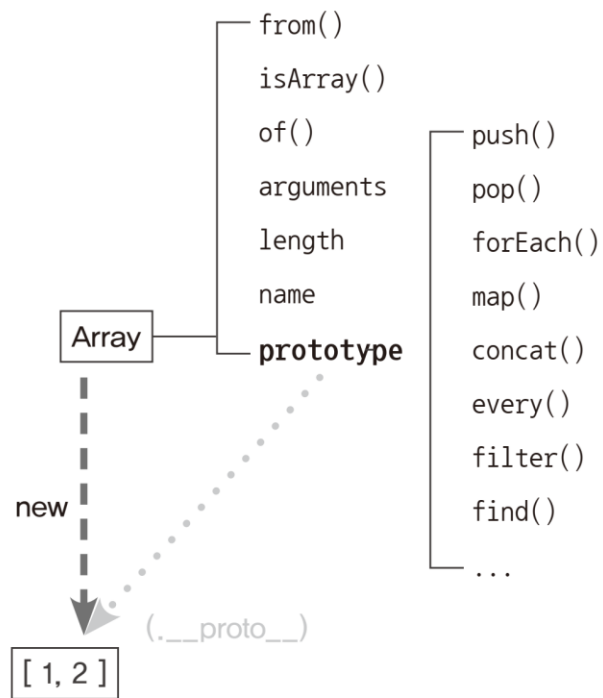
그림 6-5 크롬 개발자 도구의 콘솔에서 확인한 배열 리터럴과 Array에 대한 출력 결과(일부)

```
▼ Array(2) ⓘ
  0: 1
  1: 2
  length: 2
  __proto__: Array(0)
    ▶ concat: f concat()
    ▶ constructor: f Array()
    ▶ copyWithin: f copyWithin()
    ▶ entries: f entries()
    ▶ every: f every()
    ▶ fill: f fill()
    ▶ filter: f filter()
    ▶ find: f find()
    ▶ findIndex: f findIndex()
    ▶ flat: f flat()
    ▶ flatMap: f flatMap()
    ▶ forEach: f forEach()
    ▶ includes: f includes()
    ▶ indexOf: f indexOf()

▼ f Array() ⓘ
  arguments: (...)
  caller: (...)
  ▶ from: f from()
  ▶ isArray: f isArray()
  length: 1
  name: "Array"
  ▶ of: f of()
  ▶ prototype: Array(0)
    ▶ concat: f concat()
    ▶ constructor: f Array()
    ▶ copyWithin: f copyWithin()
    ▶ entries: f entries()
    ▶ every: f every()
    ▶ fill: f fill()
    ▶ filter: f filter()
    ▶ find: f find()
    ▶ findIndex: f findIndex()
    ▶ flat: f flat()
    ▶ flatMap: f flatMap()
    ▶ forEach: f forEach()
    ▶ includes: f includes()
    ▶ indexOf: f indexOf()
```

프로토타입이란?

그림 6-6 배열 리터럴과 Array의 관계



```
1
2  var arr = [1, 2];
3  arr.forEach(function() {});
4  Array.isArray(arr);
5  arr.isArray();
6
```

```
arr.isArray();
```

^

TypeError: arr.isArray is not a function

constructor 속성

```
1
2  var arr = [1, 2];
3  console.log(Array.prototype.constructor === Array);
4  console.log(arr.__proto__.constructor === Array);
5  console.log(arr.constructor === Array);
6
7  var arr2 = new arr.constructor(3, 4);
8  console.log(arr2);
9
```

- ➡ constructor 속성은 읽기 전용 속성이 부여된 예외적인 경우 (기본형 리터럴 변수 - number, string, boolean)를 제외하고는 값을 바꿀 수 있음


```

1
2   var NewConstructor = function () {
3       | console.log('this is new constructor!');
4   };
5
6   var dataTypes = [
7       | 1,
8       | 'test',
9       | true,
10      | {},
11      | [],
12      | function () { },
13      | /test/,
14      | new Number(),
15      | new String(),
16      | new Boolean(),
17      | new Object(),
18      | new Array(),
19      | new Function(),
20      | new RegExp(),
21      | new Date(),
22      | new Error(),
23  ];
24
25  dataTypes.forEach(function (d) {
26      | d.constructor = NewConstructor;
27      | console.log(d.constructor.name, '&', d instanceof NewConstructor);
28  });
29
30

```

[illegible]

constructor 속성

```
1
2  var Person = function (name) {
3    |   this.name = name;
4  };
5
6  var p1 = new Person('사람1');
7  var p1Proto = Object.getPrototypeOf(p1);
8  var p2 = new Person.prototype.constructor('사람2');
9  var p3 = new p1Proto.constructor('사람3');
10 var p4 = new p1.__proto__.constructor('사람4');
11 var p5 = new p1.constructor('사람5');
12
13 [p1, p2, p3, p4, p5].forEach(function (p) {
14 |   console.log(p, p instanceof Person);
15 });
16
```

```
Person { name: '사람1' } true
Person { name: '사람2' } true
Person { name: '사람3' } true
Person { name: '사람4' } true
Person { name: '사람5' } true
```

메소드 오버라이드

```
1
2  var Person = function (name) {
3    |   this.name = name;
4  };
5
6  Person.prototype.getName = function () {
7    |   return this.name;
8  };
9
10 var iu = new Person('지금');
11
12 iu.getName = function () {
13   |   return '바로 ' + this.name;
14 };
15
16 console.log(iu.getName());
17
```

```
17
18 console.log(iu.__proto__.getName());
19
```

```
19
20 console.log(iu.__proto__.getName.call(iu));
21
```

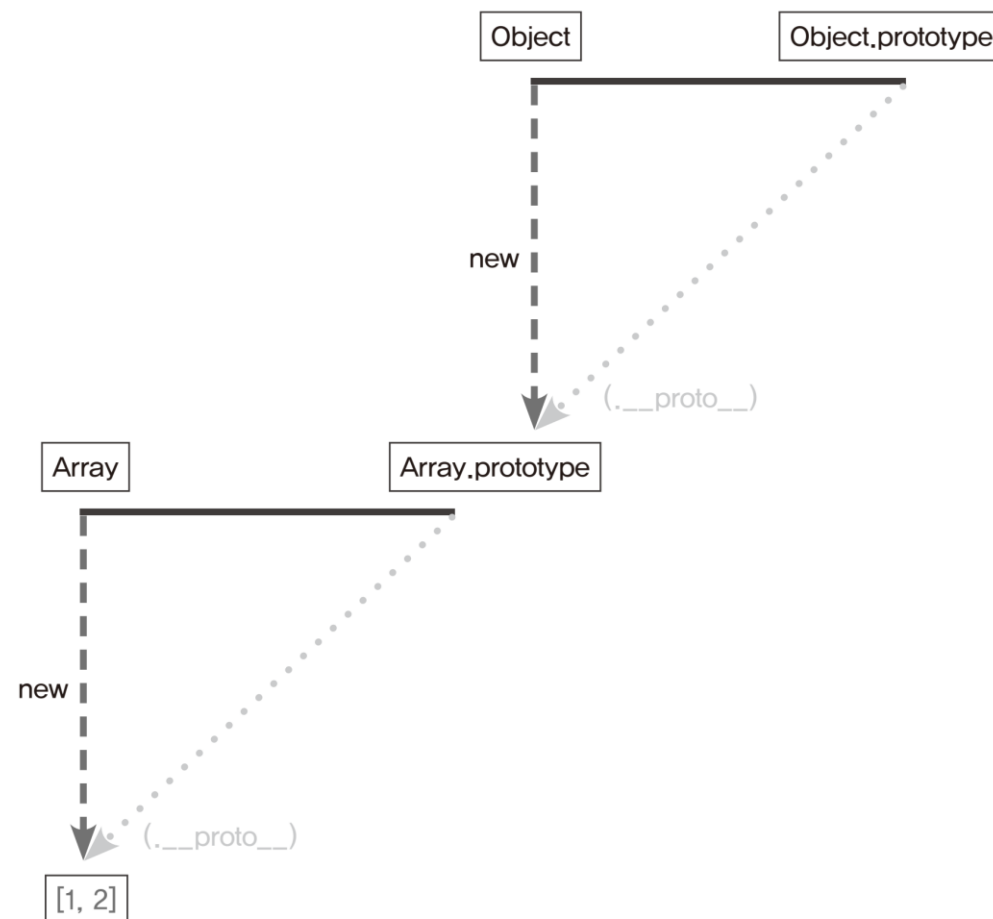
프로토타입 체인

```
1  
2   var arr = [1, 2];  
3   arr.push(3);  
4   arr.hasOwnProperty(2);  
5
```

그림 6-8 배열의 내부 구조

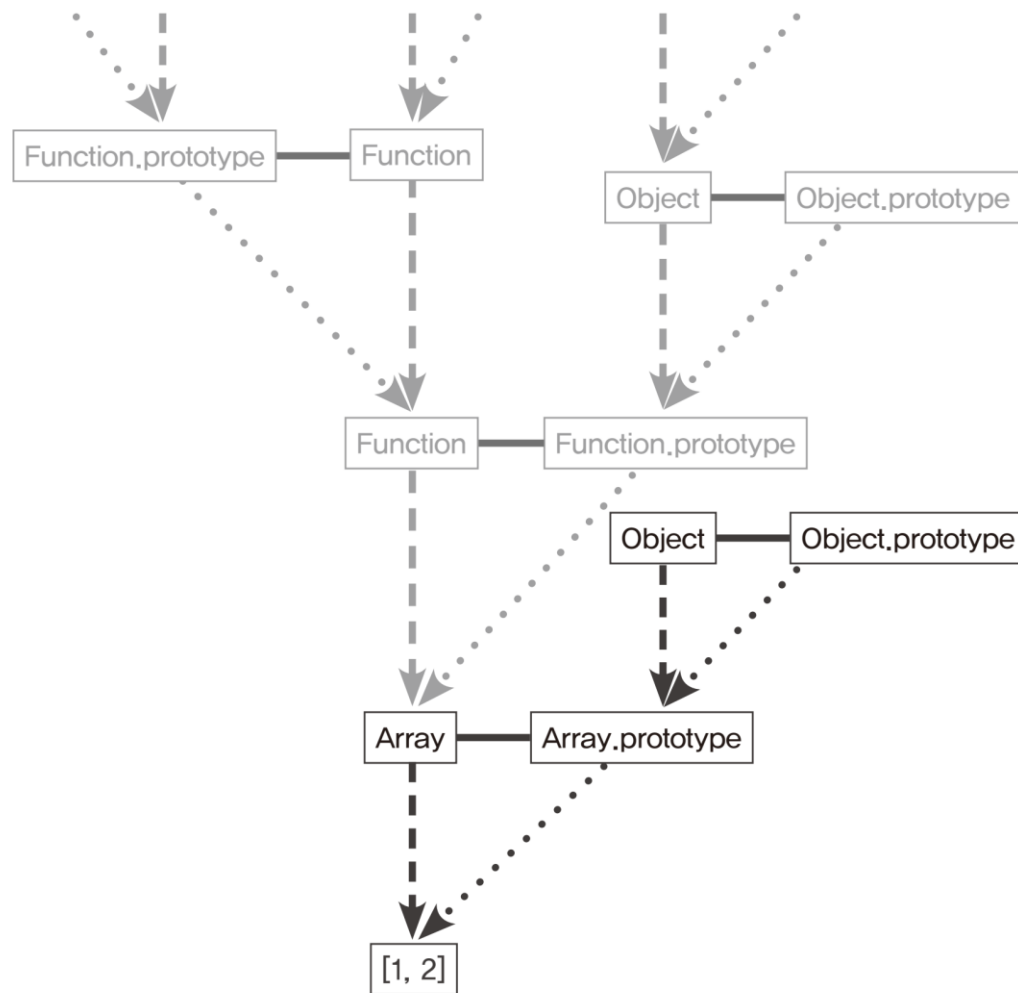
```
▼ Array(2) 1  
  0: 1  
  1: 2  
  length: 2  
  ▼ __proto__: Array(0)  
    ▶ concat: f concat()  
    ▶ constructor: f Array()  
    ▶ copyWithin: f copyWithin()  
  ----- 종략 -----  
  ▶ values: f values()  
  ▶ Symbol(Symbol.iterator): f values()  
  ▶ Symbol(Symbol.unscopables): {copyWithin:  
  ▼ __proto__:  
    ▶ constructor: f Object()  
    ▶ hasOwnProperty: f hasOwnProperty()  
    ▶ isPrototypeOf: f isPrototypeOf()  
    ▶ propertyIsEnumerable: f propertyIsEnumerable()  
    ▶ toLocaleString: f toLocaleString()  
    ▶ toString: f toString()  
    ▶ valueOf: f valueOf()  
    ▶ __defineGetter__: f __defineGetter__()  
    ▶ __defineSetter__: f __defineSetter__()  
    ▶ __lookupGetter__: f __lookupGetter__()  
    ▶ __lookupSetter__: f __lookupSetter__()  
    ▶ get __proto__: f __proto__()  
    ▶ set __proto__: f __proto__()
```

그림 6-9 배열의 내부 도식



프로토타입 체인

그림 6-11 전체 프로토타입 구조



Object.create(null)

- ▶ 예외적으로 Object.prototype의 메소드의 접근할 수 없는, __proto__ 속성이 없는 객체를 생성
- ▶ 내장 메소드 및 속성들이 제거되지만 객체 자체의 무게가 가벼움으로 성능상 이점을 가짐

```
1
2  var proto = Object.create(null);
3  console.dir(proto);
4
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe ex.js
[Object: null prototype] {}

다중 프로토타입 체인

그림 6-14 다중 프로토타입 체이닝(2)

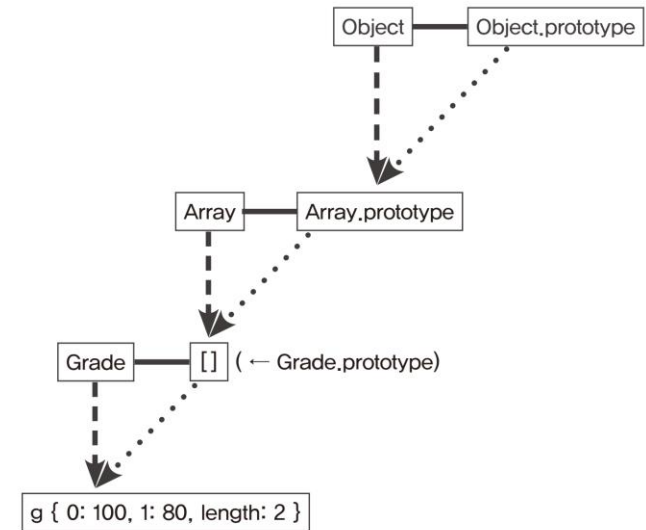
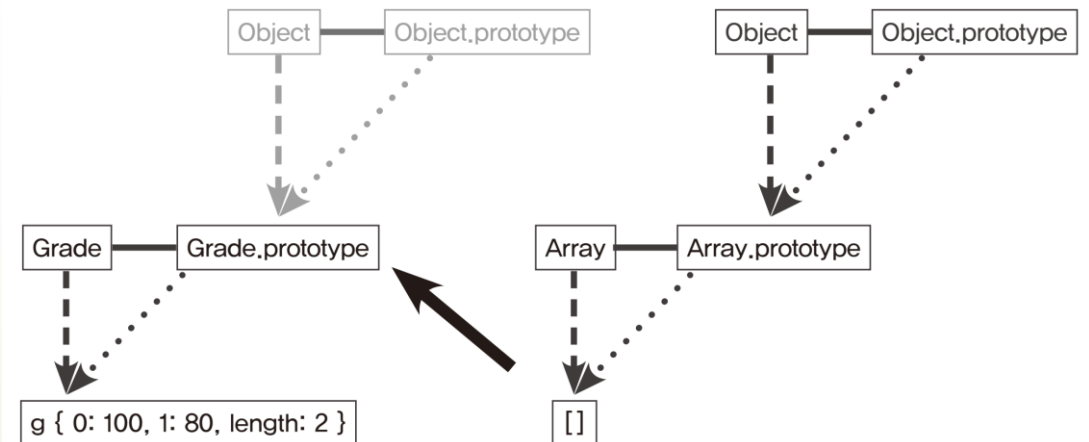
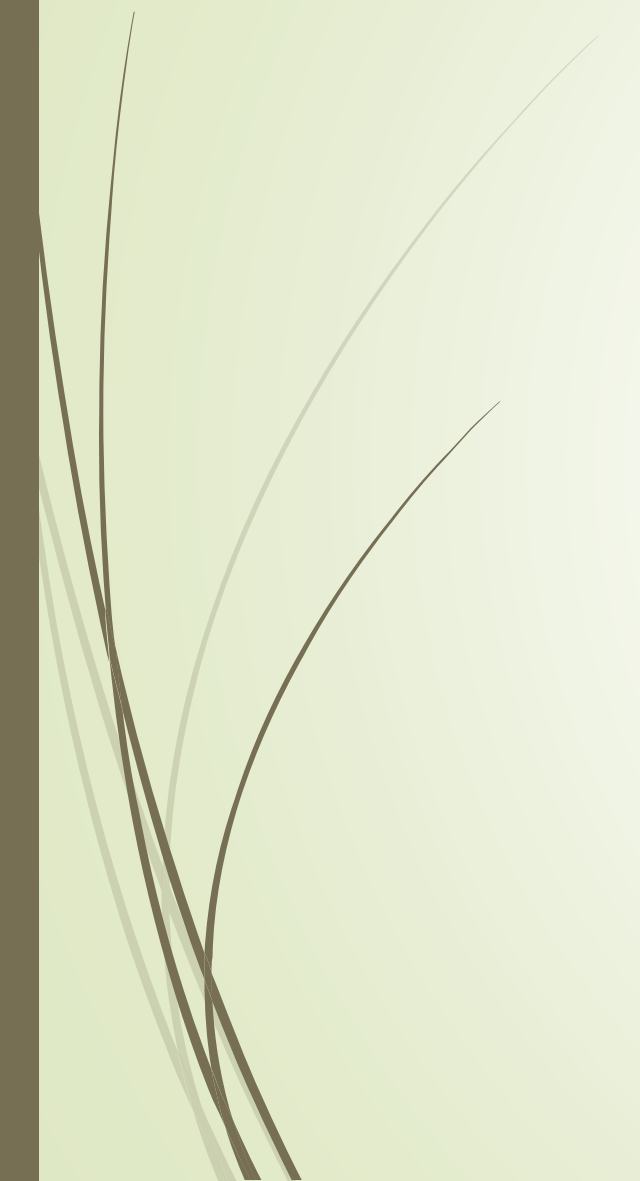


그림 6-13 다중 프로토타입 체이닝(1)



```
1
2 var Grade = function () {
3     var args = Array.prototype.slice.call(arguments);
4     for (var i = 0; i < args.length; i++) {
5         this[i] = args[i];
6     }
7     this.length = args.length;
8 };
9
10 Grade.prototype = []
11
12 var g = new Grade(100, 80);
13
14 console.log(g);
15 g.pop();
16 console.log(g);
17 g.push(90);
18 console.log(g);
19
```



Thank you!